

Arquitetura e Ferramentas Sugeridas para Pré-Matrícula

Contexto: o problema de escalonamento de turmas (timetabling) é classicamente NP-difícil ¹. Assim, o ideal é usar um *solver* de otimização capaz de lidar com muitas restrições. A título de comparação, bibliotecas como PuLP e Pyomo são linguagens de modelagem MIP em Python, enquanto o Google OR-Tools oferece um conjunto integrado de solvers (CP-SAT, programação linear, roteirização, etc.) ² ³.

Escolha do Solver de Otimização

- **Google OR-Tools (CP-SAT)** – recomendo fortemente para este caso. É open-source, escrito em C++ com interface Python, e inclui o solver CP-SAT que combina Programação por Restrições e SAT, ótimo para problemas de escalonamento ². O OR-Tools também possui tutoriais de *job shop scheduling* e é usado em desafios de timetabling. Por ser de alta performance e gratuito, atende bem a demandas grandes.
- **Pyomo** – é uma linguagem de modelagem rica para LP/MIP que permite definir modelos complexos e depois escolher vários solvers (GLPK, CBC, CPLEX, Gurobi etc.) ³. Pyomo é poderosa e flexível para modelos grandes e multifacetados, mas exige mais código e tem curva de aprendizado maior ³. Pode ser usada se preferir formular o problema como MIP clássico.
- **PuLP** – biblioteca Python simples para LP/MIP, ótima para protótipos e problemas pequenos ⁴. No entanto, seu desempenho cai em modelos muito grandes ou com muitas restrições combinatórias (como evitar conflitos de horário), tornando-o menos indicado para um cronograma complexo.

Em resumo, **OR-Tools CP-SAT** é a opção mais robusta para escalonamento de turmas, dada sua especialização em restrições e sua eficiência comprovada ² ¹. Pyomo é alternativa viável se quiser usar um solver de programação inteira padrão, mas requer solvers externos; PuLP seria apenas para protótipos ou escalonamento simples.

Framework Web/API (Backend)

- **FastAPI** – framework moderno e de alto desempenho para construir APIs REST em Python ⁵. É assíncrono (baseado em *Starlette*) e usa **type hints** do Python, gerando automaticamente documentação OpenAPI interativa. O FastAPI é conhecido por ser “*muito rápido*” e fácil de usar, reduzindo tempo de desenvolvimento ⁵. Ele é ideal para um serviço de agendamento que receba dados (p. ex., CSV de pré-matrícula) e dispare o solver em segundo plano.
- **Flask** – microframework minimalista e maduro, simples de aprender (baseado em WSGI) ⁶. É excelente para aplicações pequenas ou protótipos; porém, não é assíncrono por padrão e requer mais configuração manual (outras extensões) para APIs complexas. Flask não gera documentação automática como o FastAPI, e seu desempenho é menor em tarefas muito concorrentes.

Para um sistema de pré-matrícula moderno, **FastAPI** costuma ser a escolha recomendada (rapidez e produtividade) ⁵. Porém, se a equipe preferir algo mais tradicional e simples, **Flask** também funciona, ficando mais lento em cenários de alta carga. Ambos podem ser facilmente empacotados com Unicorn/Gunicorn em produção.

Modelagem de Dados Acadêmicos

Sugere-se um **modelo de dados** que reflita salas, turmas, professores e horários (timeslots):

- **Sala** – atributos: `id`, `nome`, `capacidade`, `tem_computador` (booleano).
- **Turma** – atributos: `id`, `codigo`, `numero_alunos`, `precisa_computador` (booleano), `professor_id`, etc. A turma “precisa_computador” indica laboratórios de TI e só pode ir em salas com `tem_computador = true`.
- **Professor** – atributos: `id`, `nome`, possivelmente `disciplinas_que_ministra`.
- **Horário (Timeslot)** – objeto com dia da semana e período (ex.: “2M1” para 2ª/Manhã horário 1), ou representação equivalente. Cada timeslot pode corresponder a pares (ex.: M12, M34) conforme descrição.
- **Preferência do Aluno (Pré-matricula)** – lista de possíveis turmas desejadas e horários preferidos. Usaremos isso para inferir demanda e evitar conflitos de horários simultâneos nas alocações.

Na implementação, podemos usar **SQLAlchemy (ORM)** para mapear essas entidades no Python. Por exemplo, em `models.py` criar classes como abaixo:

```
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey
from sqlalchemy.orm import relationship
from database import Base

class Sala(Base):
    __tablename__ = "salas"
    id = Column(Integer, primary_key=True)
    nome = Column(String, unique=True)
    capacidade = Column(Integer)
    tem_computador = Column(Boolean)

class Turma(Base):
    __tablename__ = "turmas"
    id = Column(Integer, primary_key=True)
    codigo = Column(String)
    numero_alunos = Column(Integer)
    precisa_computador = Column(Boolean)
    professor_id = Column(Integer, ForeignKey("professores.id"))
    professor = relationship("Professor")
    # outros campos...
```

Em conjunto, vamos criar *endpoints* na API (FastAPI/Flask) para importar dados (CSV ou JSON) de turmas, professores, preferências etc., e armazenar no banco de dados.

Uso de Dados Abertos da UFRN

A UFRN disponibiliza **dados abertos oficiais** (grade de horários, disciplinas, salas, professores) que podem ser usados para povoar o sistema ⁷. Há projetos de exemplo (como *SuggestClasses* da UFRN) que baixam CSVs públicos para preencher tabelas de horários, centros, salas, departamentos e componentes curriculares ⁷. Sugerimos aproveitar esses CSVs ou APIs abertas do MEC/UFRN para

obter: lista de cursos e turmas por período, docentes vinculados às turmas, salas disponíveis (com/sem computador) e horários letivos padrão.

Além disso, os **questionários de expectativa de matrícula** (pré-matrícula) devem gerar um arquivo de preferências dos alunos (p.ex., CSV com “aluno_id, turma_desejada, prioridade, horários_preferidos”). Esses dados alimentam o solver para ajustar a oferta de turmas de acordo com a demanda real.

Estrutura de Projeto e Arquivos Iniciais

Uma estrutura de pastas sugerida para Python/FastAPI é:

```
pre-matricula/
├─ app/
│   ├── main.py          # inicializa FastAPI e rota principal
│   ├── models.py        # entidades (SQLAlchemy)
│   ├── routes.py        # endpoints (importação dados, consulta)
│   └── solver.py         # lógica de montagem do modelo e chamada ao OR-Tools
├─ data/                 # scripts de importação e amostras de CSV
│   ├── importar_ufrn.py
│   └── exemplo_pre_matricula.csv
├─ requirements.txt      # dependências
└─ README.md
```

- **main.py:** instancia `FastAPI()`, configura rotas de API.
- **models.py:** define classes ORM (Sala, Turma, Professor, Timeslot, PreferenciaAluno).
- **routes.py:** endpoints REST, ex.: `POST /importar-turmas` para ler CSV e gravar no BD.
- **solver.py:** monta problema de otimização (usa OR-Tools) com todas as restrições (capacidade, conflito de sala/horário, necessidade de laboratórios) e executa o solver.

Exemplo de requirements.txt

```
fastapi==0.100.0
uvicorn[standard]==0.22.0
sqlalchemy==1.4.0
psycpg2-binary==2.9.0
pandas==2.0.0
ortools==9.8.18613
```

Esses pacotes incluem o FastAPI (com Uvicorn para o servidor), SQLAlchemy e driver PostgreSQL, Pandas para manipular CSV e OR-Tools para otimização. Ajuste as versões conforme atualizações recentes.

Exemplo de main.py (FastAPI)

```
from fastapi import FastAPI
from sqlalchemy.orm import Session
from database import engine, Base
```

```
# Inicializa DB (cria tabelas)
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Pré-Matrícula UFRN")

@app.get("/")
def raiz():
    return {"mensagem": "API de Pré-Matrícula em funcionamento"}

# Aqui incluir rotas para importar dados, iniciar solver etc.
```

Conclusão

Em resumo, uma pilha **Python** recomendada para este projeto seria: FastAPI (backend), SQLAlchemy+PostgreSQL (persistência), e Google OR-Tools (solver de escalonamento) ² ⁵. Essa combinação é leve e focada em APIs, permitindo construir rapidamente um serviço que importa dados acadêmicos (usando dados abertos da UFRN) e executa otimização para distribuir turmas conforme a demanda estudantil. Conforme previsto, Ferramentas como OR-Tools são apropriadas para problemas NP-difíceis de timetabling ¹, e FastAPI traz desempenho e facilidade para expor o agendador como um serviço web. A estrutura de arquivos acima e o requirements servem de ponto de partida; a partir deles, pode-se iterar e ajustar peso de restrições ou escalonar testes unitários conforme a evolução do projeto.

Fontes: Documentação e comparativos de OR-Tools, Pyomo e FastAPI ² ³ ⁵ ⁶, além de exemplo de uso de dados abertos da UFRN ⁷ e nota sobre a complexidade NP-difícil do timetabling ¹.

¹ OptaPlanner User Guide

https://docs.optaplanner.org/latest/optaplanner-docs/html_single/index.html

² ³ ⁴ MIP Solvers Unleashed: A Beginner's Guide to PuLP, CPLEX, Gurobi, Google OR-Tools, and Pyomo | by Ashutosh Shenoy | Operations Research Bit | Medium

<https://medium.com/operations-research-bit/mip-solvers-unleashed-a-beginners-guide-to-pulp-cplex-gurobi-google-or-tools-and-pyomo-0150d4bd3999>

⁵ FastAPI

<https://fastapi.tiangolo.com/>

⁶ Welcome to Flask — Flask Documentation (3.1.x)

<https://flask.palletsprojects.com/en/stable/>

⁷ GitHub - labens-ufrn/suggestclasses: SuggestClasses - Sistema de Sugestão de Horários para o BSI/UFRN.

<https://github.com/labens-ufrn/suggestclasses>